

PROOF OF CONCEPT

AI Meeting Summarization Assistant

An Enterprise RAG-Based AI Assistant for Meeting Transcription, Summarization & Action-Item Intelligence

Document Code: POC-MEET-004

Classification: Internal — AI Mentor Review

Prepared by: AI Solutions Architecture Team

Version 1.0 — POC Approval Draft

Table of Contents

1. Executive Summary
2. Problem Statement
3. Proposed AI Solution
4. System Architecture
5. Complete Folder Structure
6. Functional Modules
7. APIs
8. Database Design
9. RAG Pipeline
10. LangChain + LangGraph Flow
11. Tech Stack
12. Production Features
13. Future Enhancements
14. Risks
15. Development Roadmap
16. Conclusion

1. Executive Summary

1.1 The Business Problem

Organizations run large volumes of recurring meetings whose decisions, commitments, and context are captured inconsistently, if at all. Action items are lost in scattered personal notes, decisions made in one meeting are not visible to stakeholders who missed it, and answering 'what did we decide about X' requires manually searching through recordings or asking around, often unsuccessfully.

1.2 Why This Problem Persists

Meeting content is inherently unstructured spoken conversation, captured (if at all) as long, unindexed video/audio recordings or inconsistent personal notes, with no organization-wide system that reliably extracts, structures, and makes searchable the decisions and commitments made across meetings.

1.3 Why Current Solutions Fall Short

Basic call-recording tools provide only a raw transcript or a generic single-paragraph summary; they cannot answer specific follow-up questions, cannot track whether an action item was later resolved, and cannot connect a decision made in one meeting to related discussion in another.

1.4 Why AI + Retrieval-Augmented Generation (RAG)

RAG allows employees to ask specific, natural-language questions across the entire history of meeting transcripts and receive a grounded answer with a citation to the exact meeting, timestamp, and speaker, rather than needing to re-watch recordings or rely on a single static summary.

1.5 Business Value

- Eliminates time spent searching recordings to recall what was decided or committed to.
- Improves accountability by surfacing and tracking action items automatically.
- Gives absent stakeholders reliable, cited access to what was discussed and decided.
- Reduces redundant re-litigation of decisions already made in a prior meeting.
- Creates a searchable institutional memory of project and team decisions over time.

1.6 Target Users

Persona	Role	Primary Need
Project Manager	Tracks decisions and action items across meetings	Automated action-item extraction and tracking
Team Member	Missed a meeting or needs to recall detail	Cited answers without re-watching recordings
Executive	Needs cross-meeting visibility into project status	Synthesized cross-meeting summaries
Team Lead	Runs recurring meetings and needs follow-up	Reliable capture of commitments and owners

1.7 Expected Impact

- Target: significant reduction in time spent searching for prior meeting context.
- Target: measurable increase in action-item completion rate through automated tracking.
- Reduced redundant discussion of previously settled decisions.
- Improved stakeholder alignment for team members unable to attend live.

2. Problem Statement

2.1 Detailed Problem Description

Teams increasingly operate across distributed, asynchronous schedules, making it common for stakeholders to miss meetings relevant to their work. Even for attendees, human memory of specifics — exact wording of a decision, who owns a given action item, what was agreed as a deadline — degrades quickly, and no systematic mechanism exists to preserve and retrieve that information reliably.

2.2 Industry Examples

Organization Type	Scenario	Consequence
Product organization	A design decision made in a planning meeting is not communicated to an engineer who was on leave, leading to rework.	Two weeks of engineering effort built against the wrong requirement.
Sales organization	A verbal commitment made to a customer during a call is not logged anywhere and is forgotten by the account team.	Customer escalation over an unmet commitment with no record of what was promised.
Executive leadership team	A strategic decision is revisited and re-debated in a later meeting because no one can recall the original reasoning.	Wasted leadership time re-litigating settled decisions.

2.3 Current Workflow (As-Is)

1. Meeting occurs, optionally recorded by a conferencing tool.
2. Attendees take personal, inconsistent notes or rely on memory.
3. If a summary is produced, it is a generic auto-summary with no structured action items or citations.
4. Recording is stored in a call archive that is rarely, if ever, revisited.
5. Someone later asks 'what did we decide about X' and must manually search recordings or ask colleagues.
6. Action items, if captured, live in disconnected personal to-do lists with no centralized tracking.

2.4 Pain Points

- No structured, searchable record of decisions and commitments made in meetings.
- Action items are frequently lost or forgotten with no systematic tracking.
- Absent stakeholders have no reliable way to catch up on relevant discussion.
- Re-litigation of previously settled decisions wastes meeting time.
- Manually reviewing recordings to find a specific moment is slow and rarely done.

2.5 Compliance, Regulatory & Documentation Challenges

Meeting recordings and transcripts may contain sensitive business, personnel, or customer information, and in some jurisdictions recording consent and retention rules apply. The system must respect organizational data-

retention policies, restrict access to meeting content based on original attendee/invitee permissions, and support deletion requests in line with applicable data protection requirements.

3. Proposed AI Solution

3.1 Solution Overview

The AI Meeting Summarization Assistant transcribes meeting audio, structures the transcript into a searchable, permission-scoped knowledge base, automatically extracts decisions and action items, and allows any authorized employee to ask natural-language questions across meeting history with answers cited to the exact meeting and timestamp.

3.2 Why Retrieval-Augmented Generation Is the Correct Approach

Because the value of the system depends on precisely locating what was said in a specific meeting, retrieval against an indexed, timestamped transcript corpus is required; a model without access to the actual transcript content cannot know what was discussed or decided in any given meeting.

3.3 Why a Standard Chatbot Is Not Sufficient

A generic chatbot has no access to an organization's private meeting recordings and, without retrieval, could only offer generic advice about running meetings rather than answering specific questions about what a particular meeting actually covered.

3.4 Why Vector Search Is Required

Questions about meeting content are naturally phrased around topics and intent ('what did we decide about the pricing change') rather than exact transcript wording, so semantic search is needed to match the question to the relevant transcript segment, while metadata filtering narrows results to the correct meeting, date range, or project.

3.5 How the LLM Improves the Workflow

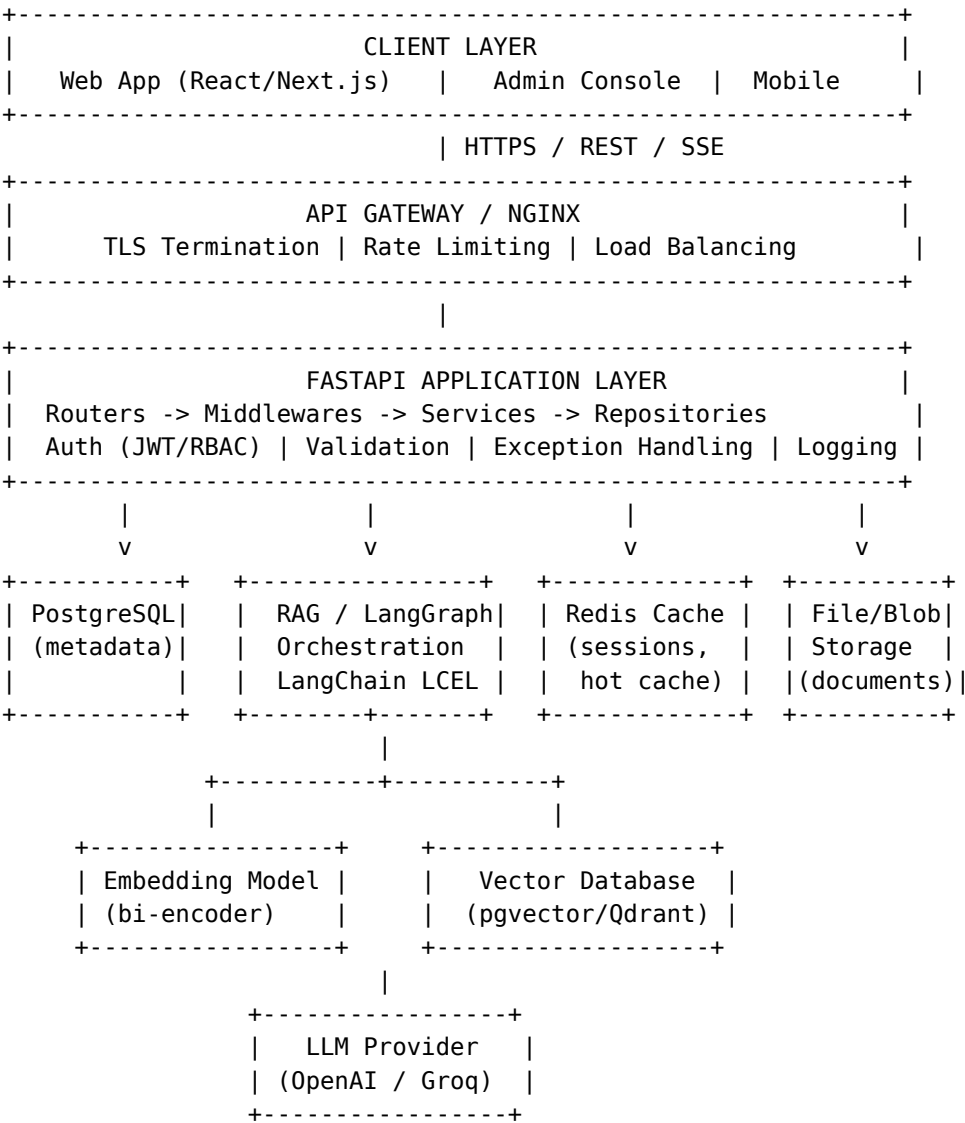
The LLM synthesizes relevant transcript segments — potentially spanning multiple meetings — into a coherent answer or summary, extracts structured action items with inferred owners and deadlines, and generates cross-meeting status rollups that would otherwise require manually reviewing multiple recordings.

4. System Architecture

4.1 Architectural Overview

The AI Meeting Summarization Assistant platform is built as a modular, cloud-deployable, layered architecture. Each layer has a single responsibility, communicates through well-defined contracts, and can be scaled or replaced independently. This separation of concerns is essential for an enterprise system that must evolve as meeting knowledge requirements, models, and regulations change over time.

4.2 High-Level Component Diagram



4.3 Component Responsibilities

Component	Responsibility
Frontend	Chat UI, document upload, admin dashboards, citation rendering, streaming response display.
API Gateway / Nginx	TLS termination, reverse proxy, load balancing, request throttling.

Component	Responsibility
FastAPI Backend	Async REST API layer; routes requests to services; enforces auth and validation.
Authentication (JWT)	Issues and verifies signed access/refresh tokens; short-lived access, rotating refresh.
Role-Based Access Control	Maps users to roles (Admin, Reviewer, Analyst, Viewer) and enforces per-endpoint scopes.
LangChain (LCEL)	Declarative composition of prompt -> retriever -> LLM -> parser as runnables.
LangGraph	Stateful orchestration: conditional routing, retries, human-approval nodes, multi-step agents.
Document Parser	Extracts text/tables from PDF, DOCX, TXT, HTML while preserving structure and page numbers.
Chunking Engine	Splits parsed text into semantically coherent chunks with overlap and metadata tags.
Embedding Model	Converts chunks and queries into dense vectors for semantic similarity search.
Vector Database	Stores embeddings and metadata; supports ANN search and hybrid (keyword + vector) queries.
Retriever	Fetches top-k relevant chunks per query, applies metadata filters (date, doc type, role).
Reranker	Cross-encoder re-scoring of retrieved chunks to improve precision before context injection.
Prompt Template	Structured, versioned prompts with system instructions, guardrails, and citation formatting.
Output Parser	Validates and structures LLM output (JSON schema, citation extraction, refusal detection).
LLM	Generates grounded natural-language answers strictly from the retrieved context.
PostgreSQL	System of record for users, roles, documents, conversations, audit logs, analytics.
Redis	Session cache, rate-limit counters, short-term conversation memory, embedding cache.
File / Blob Storage	Durable storage of original source documents and processed artifacts.
Logging & Monitoring	Structured logs, metrics, tracing across every request for observability and audit.
CI/CD	Automated build, test, container image publishing, and deployment pipeline.

4.4 End-to-End Data Flow

The following sequence describes how a single user query flows through the system, from ingestion of source documents to the delivery of a grounded, cited answer:

- Documents are uploaded via the Document Upload API and stored in Blob Storage.
- The Document Parser extracts raw text and structural metadata (page, section, table).

9. The Chunking Engine splits content into overlapping chunks and tags each with source metadata.
10. The Embedding Model converts each chunk into a vector, which is written to the Vector Database alongside metadata.
11. A user submits a natural-language question through the Chat API.
12. The query is embedded and passed to the Retriever, which performs a hybrid (vector + keyword) search filtered by the user's role and permitted document scope.
13. The Reranker re-orders the top candidate chunks by relevance to the exact query.
14. LangChain (LCEL) assembles a Prompt Template containing the system instructions, the reranked context, and conversation memory.
15. LangGraph routes the request: if confidence is low or the query touches a sensitive workflow, it is routed to a human-approval node before finalization.
16. The LLM generates a grounded response with inline citations back to source chunks.
17. The Output Parser validates the response structure and citation integrity before it is returned to the client.
18. The interaction, retrieved sources, and response are persisted to PostgreSQL for audit and analytics; the response is streamed to the client over SSE.

5. Complete Folder Structure

5.1 Repository Layout

```

src/
  app/
    routers/          # FastAPI route definitions grouped by domain (auth, chat, documents,
admin)                # Business logic orchestrating repositories, RAG pipeline, and external
    services/          # Data access layer – encapsulates all PostgreSQL/ORM queries
calls                 # DB session management, connection pooling, Alembic migrations
    repositories/      # SQLAlchemy ORM models (User, Document, Conversation, AuditLog, etc.)
    database/          # Pydantic request/response schemas (DTOs) with validation rules
    models/            # Auth, logging, rate-limiting, correlation-id, exception-handling
    schemas/           # Environment-based settings (Pydantic Settings), secrets loading
    middlewares/       # Security utilities (JWT, hashing), constants, dependency injection
middleware           # Generic helpers (pagination, formatting, retry decorators)
    config/            # Versioned prompt templates (system, retrieval, citation,
    core/              # Embedding client wrapper, batching, caching logic
wiring               # Vector DB client, index management, hybrid-search implementation
    utils/             # Parsers for PDF/DOCX/TXT, chunking strategies
    prompts/           # Retriever, reranker, context-builder, citation-formatter
summarization)        # StateGraph definitions, node functions, conditional edges, agent
    embeddings/        # Unit, integration, and end-to-end tests (pytest)
    vectorstore/       # Local log output (structured JSON, rotated)
    documents/         # FastAPI application entrypoint / ASGI app factory
    rag/               # Database migration scripts
    langgraph/         # Dockerfile(s), docker-compose for local/dev stacks
flows                # CI/CD pipeline definitions (GitHub Actions)
    tests/             # React/Next.js client application
    logs/              # Chat window, citation cards, upload widget, admin tables
    main.py            # Route-level pages
alembic/              # Data-fetching and auth hooks
docker/               # Documented environment variable template
.github/workflows/    # Python dependency and tooling configuration
frontend/             # Architecture overview and setup instructions
  src/components/
  src/pages/
  src/hooks/
.env.example
pyproject.toml
README.md

```

5.2 Purpose of Each Folder

Folder	Purpose
routers/	Thin HTTP layer; parses requests, calls services, returns schemas. No business logic.
services/	Coordinates use cases (e.g. 'ask a question', 'ingest a document'); the core business logic.
repositories/	Isolates all SQL/ORM access so services never talk to the database directly.
database/	Engine/session setup, connection pooling, and Alembic migration configuration.
models/	Declarative ORM entities mapped to PostgreSQL tables.

Folder	Purpose
schemas/	Pydantic models defining and validating API request/response contracts.
middlewares/	Cross-cutting concerns applied to every request: auth, logging, rate limits.
config/	Centralized, environment-aware configuration (dev/staging/prod).
core/	Security primitives, dependency injection, shared constants.
utils/	Reusable helper functions not tied to a specific domain.
prompts/	Prompt templates kept out of code, versioned, and independently testable.
embeddings/	Wraps the embedding provider; adds batching, retries, and caching.
vectorstore/	Abstracts the vector database so the provider can be swapped with minimal change.
documents/	Format-specific parsing and chunking logic.
rag/	The retrieval-augmented generation pipeline: retrieve, rerank, build context, cite.
langgraph/	Stateful multi-step orchestration graphs and node implementations.
tests/	Automated test suite covering unit, integration, and RAG-quality evaluation.

6. Functional Modules

The platform is decomposed into the following functional modules. Each module is independently testable and maps to a bounded context within the meeting knowledge domain.

Authentication

SSO login with JWT sessions scoped to each employee's meeting invitee/attendee permissions.

User Management

Syncs org structure and meeting attendance records to drive access scoping.

Admin Dashboard

Transcription pipeline health, storage usage, and retention policy oversight.

Document Upload

Ingestion of meeting audio/video and existing transcripts from conferencing tools.

Document Processing

Speech-to-text transcription, speaker diarization, and timestamp alignment.

Embedding Pipeline

Chunking and vectorization of transcript segments with speaker and timestamp metadata.

Vector Database

Permission-scoped store enabling retrieval filtered by meeting, project, and date range.

Chat Module

Conversational interface answering questions with citations to meeting, timestamp, and speaker.

Conversation Memory

Retains context for multi-turn follow-up questions within a research session.

Search

Direct search across transcript history independent of the chat flow.

Audit Logs

Access logging for sensitive meeting content in line with data-protection requirements.

Notifications

Automated reminders to action-item owners and status digests to meeting organizers.

Analytics

Dashboards on action-item completion rates and recurring discussion themes.

Reports

Exportable structured meeting summaries and action-item logs.

Settings

Retention period, recording-consent, and notification preferences.

Role Management

RBAC ensuring meeting content is visible only to original attendees/invitees and admins.

7. APIs

7.1 API Design Principles

- RESTful resource-oriented URLs (e.g. /api/v1/documents/{id}).
- Consistent envelope for success and error responses.
- Cursor and offset pagination supported on all list endpoints via ?page & ?page_size.
- Filtering via query parameters (?status=, ?doc_type=, ?date_from=, ?date_to=).
- Sorting via ?sort_by= and ?order=asc|desc.
- Idempotent PUT/DELETE; partial updates via PATCH where applicable.
- All endpoints versioned under /api/v1/ to allow non-breaking evolution.

7.2 Core Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate user, issue JWT access + refresh tokens.
POST	/api/v1/auth/refresh	Exchange a valid refresh token for a new access token.
GET	/api/v1/users/me	Return the authenticated user's profile and role.
POST	/api/v1/documents	Upload a new source document for ingestion.
GET	/api/v1/documents	List documents with pagination, filtering, and sorting.
GET	/api/v1/documents/{id}	Retrieve a single document's metadata and processing status.
DELETE	/api/v1/documents/{id}	Soft-delete a document and its vector embeddings.
POST	/api/v1/chat/sessions	Create a new conversation session.
POST	/api/v1/chat/sessions/{id}/messages	Submit a question; returns a streamed, cited answer.
GET	/api/v1/chat/sessions/{id}	Retrieve full conversation history.
GET	/api/v1/audit-logs	List audit log entries (Admin only), paginated and filterable.
GET	/api/v1/analytics/usage	Aggregate usage metrics for dashboards.

7.3 Standard Response Envelope

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "page_size": 20, "total_items": 134, "total_pages": 7 },
  "error": null
}
```

7.4 Standard Error Response

```
{
  "success": false,
  "data": null,
  "meta": null,
  "error": {
    "code": 400,
    "message": "Invalid request"
  }
}
```

```

"error": {
  "code": "DOCUMENT_NOT_FOUND",
  "message": "No document exists with the given ID.",
  "request_id": "3f9a2c11-..."
}
}

```

7.5 Sample Request / Response — Ask a Question

Request:

POST /api/v1/chat/sessions/8f21/messages

Authorization: Bearer <jwt>

Content-Type: application/json

```

{
  "message": "What did we decide about the Q3 pricing change and who owns the follow-up?"
}

```

Response:

```

{
  "success": true,
  "data": {
    "message_id": "9c31-...",
    "answer": "In the July 1st pricing review, the team agreed to a 5% increase on the enterprise tier effective Q3, with the follow-up customer communication plan owned by the Product Marketing lead, due by July 15th.",
    "citations": [
      { "source": "Meeting: Pricing Review – Jul 1, 2026 (42:10)", "page": 4 }
    ],
    "confidence": "high"
  },
  "error": null
}

```

7.6 HTTP Status Codes Used

Code	Meaning	Usage
200	OK	Successful GET/PUT/PATCH.
201	Created	Successful POST creating a resource.
204	No Content	Successful DELETE.
400	Bad Request	Validation failure on request payload.
401	Unauthorized	Missing or invalid JWT.
403	Forbidden	Authenticated but insufficient role/permission.
404	Not Found	Resource does not exist.
409	Conflict	Duplicate resource or state conflict.
422	Unprocessable Entity	Semantically invalid request (e.g. unsupported file type).
429	Too Many Requests	Rate limit exceeded.
500	Internal Server Error	Unhandled server-side exception.

8. Database Design

8.1 Core Tables

Table	Key Columns	Notes
users	id (PK), email, hashed_password, role_id (FK), created_at	One row per authenticated user.
roles	id (PK), name, permissions (JSONB)	Defines RBAC permission sets.
documents	id (PK), title, doc_type, storage_uri, status, uploaded_by (FK users), created_at	Metadata for every ingested source document.
document_chunks	id (PK), document_id (FK), chunk_index, content, token_count, page_number	Chunk-level metadata; vectors live in the vector store, keyed by chunk id.
conversations	id (PK), user_id (FK), title, created_at	One row per chat session.
messages	id (PK), conversation_id (FK), role, content, created_at	Individual turns within a conversation.
citations	id (PK), message_id (FK), document_chunk_id (FK), relevance_score	Links an answer to the exact source chunks used.
audit_logs	id (PK), user_id (FK), action, resource_type, resource_id, ip_address, created_at	Immutable record of security-relevant actions.
notifications	id (PK), user_id (FK), type, payload (JSONB), read_at	In-app notification queue.

8.2 Relationships

- users 1—N documents (uploaded_by)
- users 1—N conversations
- roles 1—N users
- documents 1—N document_chunks
- conversations 1—N messages
- messages 1—N citations
- document_chunks 1—N citations

8.3 Indexing Strategy

- B-tree index on users.email (unique) for fast login lookup.
- Composite index on document_chunks(document_id, chunk_index) for ordered retrieval.
- Index on audit_logs(created_at) to support time-bounded compliance queries.
- Index on messages(conversation_id, created_at) for chronological history retrieval.
- GIN index on documents.metadata (JSONB) for flexible attribute filtering.

8.4 Normalization

The schema is normalized to Third Normal Form (3NF) to eliminate redundant storage of user, document, and conversation attributes. Denormalized read-optimized views are introduced only where analytics dashboards

require aggregate performance, and are refreshed on a schedule rather than stored redundantly in transactional tables.

8.5 Entity-Relationship Summary

```
roles ---< users ---< documents ---< document_chunks ---< citations >--- messages ---<
conversations
users ---< audit_logs
users ---< notifications
```

9. RAG Pipeline

9.1 Document Loader

Format-specific loaders normalize heterogeneous inputs into a common internal representation preserving layout metadata (page, section heading, table boundaries):

Format	Loader Strategy
PDF	Layout-aware extraction (PyMuPDF/pdfplumber) preserving page numbers and table structure; OCR fallback for scanned pages.
DOCX	python-docx traversal preserving headings, tables, and styles for section-aware chunking.
TXT	Encoding-safe text read with paragraph-boundary detection.

9.2 Chunking

Recursive, structure-aware chunking targets 400–800 tokens per chunk with a 10–15% overlap, splitting first on section/heading boundaries, then paragraphs, then sentences, so that no chunk crosses a semantic boundary unnecessarily. Each chunk retains metadata: source document, page, section title, and effective date.

9.3 Embeddings

Chunks and queries are embedded using a bi-encoder sentence-transformer (or an API-based embedding model) into a shared vector space. Embeddings are batched and cached to control latency and cost, and are re-generated whenever the underlying model version changes.

9.4 Vector Database

Vectors are stored with associated metadata in a vector database (pgvector for tight PostgreSQL integration, or a dedicated store such as Qdrant for larger scale), enabling approximate nearest-neighbor search with metadata pre-filtering by document type, role scope, and date range.

9.5 Retriever & Hybrid Search

The retriever combines dense vector similarity with sparse keyword (BM25) search, merging results via reciprocal rank fusion. Hybrid search is critical in this domain because exact terminology, identifiers, and section numbers must be matched precisely, while semantic search captures paraphrased user intent.

9.6 Reranker

A cross-encoder reranker re-scores the top-N hybrid candidates against the exact user query, materially improving precision before the most relevant 4–8 chunks are passed into the LLM's context window.

9.7 Context Injection & Prompt Engineering

Reranked chunks are assembled into a structured context block with explicit source labels. The system prompt instructs the model to answer strictly from the provided context, to state when information is not available, and to attach a citation to every factual claim.

9.8 LLM Response & Citation

The LLM generates an answer grounded in the injected context. The Output Parser cross-checks that every citation in the response maps to an actual retrieved chunk, rejecting or flagging any answer with unverifiable references before it reaches the user.

9.9 Source References

Every answer displays the originating document title, page number, and an excerpt snippet, allowing the user to verify the answer against the primary source in one click — a requirement for any regulated or audit-sensitive workflow.

9.10 Conversation Memory

Short-term memory (the current session's turns) is held in Redis and included in the prompt for coherence. Long-term memory (summarized prior sessions) is persisted in PostgreSQL and selectively retrieved when relevant to the current query.

10. LangChain + LangGraph Flow

10.1 LCEL (LangChain Expression Language)

The pipeline is composed declaratively as a chain of Runnable: prompt | retriever | llm | output_parser. LCEL provides a uniform interface (invoke, batch, stream) across every step, making the pipeline composable, testable in isolation, and easy to reason about.

```
chain = prompt_template | retriever | reranker | llm | output_parser

chain.invoke(query)          # synchronous, single request
chain.batch([q1, q2, q3])    # parallel batch processing
await chain.ainvoke(query)    # async, single request (used in FastAPI routes)
await chain.abatch([...])    # async batch (bulk re-indexing / evaluation jobs)
```

10.2 StateGraph (LangGraph)

For workflows that require more than a single retrieve-then-generate pass, LangGraph models the interaction as a state machine. Each node performs one responsibility (classify intent, retrieve, rerank, generate, validate); edges route the state to the next node based on runtime conditions.

```
START -> classify_intent
classify_intent -> [ retrieve_context | direct_answer | escalate ]    (conditional edge)
retrieve_context -> rerank -> generate_answer
generate_answer -> validate_citations
validate_citations -> [ END | human_approval ]                      (conditional edge)
human_approval -> END
```

10.3 Conditional Edges

Conditional edges inspect the current state (confidence score, document sensitivity flag, citation completeness) and route execution accordingly — for example, sending low-confidence or high-stakes answers to a human-approval node instead of returning them directly to the user.

10.4 Agentic Flow

For multi-step questions (e.g. requiring lookups across multiple document types or a calculation step), LangGraph enables an agent loop where the model can call retrieval or calculation tools iteratively before composing a final answer, with a bounded step count to prevent runaway loops.

10.5 Memory

LangGraph's checkpointing persists the full state of a conversation, enabling pause/resume for human-approval steps and reliable recovery if a node fails mid-execution.

10.6 Fallback Strategy

If the primary LLM provider errors or times out, the graph falls back to a secondary provider or a cached response strategy, and surfaces a clear degraded-mode indicator to the user rather than failing silently.

10.7 Human-in-the-Loop Approval

For high-stakes outputs, LangGraph pauses execution at a `human_approval` node, notifying a reviewer through the Notifications module; the graph resumes only after explicit reviewer sign-off, which is itself recorded in the audit log.

11. Tech Stack

Layer	Technology	Purpose
Frontend	React / Next.js, TypeScript, Tailwind	Chat interface, document management, admin dashboards.
Backend Framework	FastAPI (Python 3.11+)	Async REST API layer.
Orchestration	LangChain (LCEL) + LangGraph	RAG pipeline composition and stateful workflow control.
Primary LLM	OpenAI GPT-4-class models	Answer generation, summarization, structured extraction.
Secondary LLM	Groq-hosted open models	Low-latency fallback and cost-sensitive workloads.
Embedding Model	Sentence-Transformers / OpenAI text-embedding	Dense vector representation of text.
Vector Database	pgvector (or Qdrant for scale)	Semantic similarity search.
Relational Database	PostgreSQL 15+	System of record for users, documents, conversations, audit.
Cache / Session Store	Redis	Caching, rate limiting, session and short-term memory.
Object Storage	S3-compatible blob storage	Durable storage of source documents.
Reverse Proxy	Nginx	TLS termination, load balancing.
Containerization	Docker	Reproducible builds and deployment artifacts.
CI/CD	GitHub Actions	Automated test, build, and deployment pipeline.
Cloud Deployment	AWS / Azure (Kubernetes or ECS)	Managed, autoscaling production hosting.
Observability	OpenTelemetry, Prometheus, Grafana	Metrics, tracing, dashboards, alerting.

12. Production Features

Feature	Implementation Detail
JWT Authentication	Short-lived signed access tokens (15 min) with rotating refresh tokens (7 days), stored httpOnly.
Role-Based Access Control	Declarative permission checks per endpoint, enforced via FastAPI dependencies.
Logging	Structured JSON logs with correlation IDs, shipped to a centralized log store.
Exception Handling	Global exception handlers mapping domain errors to consistent HTTP responses.
Monitoring	OpenTelemetry traces + Prometheus metrics on latency, error rate, and token usage.
Retry Mechanism	Exponential backoff with jitter on LLM/embedding provider calls.
Rate Limiting	Per-user and per-IP token-bucket limits enforced at the gateway and application layer.
Caching	Redis caching of embeddings, frequent queries, and session data.
Pagination	Cursor/offset pagination on every list endpoint with configurable page size.
Search	Hybrid vector + keyword search with metadata filters.
Async APIs	Fully async FastAPI routes and DB access (asyncpg / SQLAlchemy async).
Streaming Response	Server-Sent Events streaming of LLM tokens for perceived low latency.
File Validation	MIME-type, size, and malware-scan checks on every upload.
Security	Encryption in transit (TLS) and at rest, secrets management, input sanitization.
Audit Logs	Immutable, append-only log of every access and administrative action.
Scalability	Stateless API pods behind a load balancer; horizontal autoscaling on CPU/queue depth.

13. Future Enhancements

- Real-time in-meeting assistant surfacing relevant past-meeting context live.
- Automatic linking of related discussions across meetings on the same topic or project.
- Integration with project-management tools to sync extracted action items directly.
- Sentiment and engagement analytics for recurring team meetings.
- Multi-language transcription and cross-language semantic search.

14. Risks

14.1 Technical Risks

Risk	Description	Mitigation
Transcription accuracy	Speech-to-text errors distort meaning, especially with accents, cross-talk, or poor audio.	Confidence-scored transcription with manual correction workflow for low-confidence segments.
Consent and privacy	Recording or retaining meeting content without proper consent creates legal exposure.	Explicit recording-consent capture and jurisdiction-aware retention policy enforcement.
Access scope errors	A user retrieves content from a meeting they were not invited to.	Retrieval filtered strictly by verified attendee/invitee list, tested in CI.
Action-item misattribution	AI infers the wrong owner or deadline for an extracted action item.	Extracted action items require explicit owner confirmation before being tracked as committed.

14.2 AI Hallucination Risk

Because extracted decisions and action items can be treated as authoritative commitments by teams who were not present, an ungrounded or fabricated answer carries real business and compliance risk. The system mitigates this through strict context-grounding (the model is instructed never to answer outside retrieved context), mandatory citation validation, confidence scoring, and human-approval routing for high-stakes responses.

14.3 Mitigation Summary

- Retrieval-grounded generation with explicit 'answer only from context' instructions.
- Citation-integrity validation before any response reaches the user.
- Confidence thresholds that route uncertain answers to human review.
- Continuous evaluation against a labeled question/answer regression set.
- Versioned prompts and models with rollback capability.

15. Development Roadmap

Phase	Weeks	Deliverables
POC	1-3	Single-tool transcription connector; core RAG chat over transcripts with citations; internal demo.
MVP	4-8	Action-item extraction, RBAC, notifications, analytics dashboard, pilot with one team.
Production Hardening	9-11	Security/privacy review, load testing, monitoring/alerting, CI/CD pipeline, DR plan.
Deployment	12	Org-wide staged rollout with retention-policy configuration and adoption tracking.

16. Conclusion

Meeting knowledge is currently almost entirely lost after the meeting ends; a grounded RAG assistant turns that ephemeral spoken content into a durable, searchable, and actionable institutional record with clear, demonstrable value to any team running recurring meetings.

Given how universally applicable recurring meetings are across every function of an enterprise, and the direct, trackable link to action-item completion and stakeholder alignment, this project presents a compelling case for continued investment beyond the POC.

Prepared for approval review — AI Mentor sign-off requested prior to POC development kickoff.